

Callbacks de entrenamiento

Qué es un callback

Un **callback** es una función que se ejecuta durante el entrenamiento (antes o después de una época, un batch, etc.) y permite **intervenir automáticamente** en el proceso. Puedes usarlos para:

- Detener el entrenamiento si no mejora (`EarlyStopping`)
- Ajustar la tasa de aprendizaje (`ReduceLROnPlateau`)
- Guardar modelos (`ModelCheckpoint`)
- Visualizar métricas (`TensorBoard`)

Se colocan en el history de entrenamiento

```
history = model.fit(
    X_train, y_train,
    epochs=200,
    batch_size=32,
    validation_data=(X_val, y_val),
    callbacks=[early_stopping, lr_scheduler], # Añadir aquí los callbacks
    verbose=1
)
```

Callbacks que ya hemos usado

1. `EarlyStopping`

```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=20,
    restore_best_weights=True
)
```

Detiene automáticamente el entrenamiento cuando **una métrica de validación deja de mejorar**.

Evita **overfitting** y ahorra tiempo de entrenamiento innecesario.

Parámetros:

- `monitor` : Qué métrica observar. Aquí es `'val_loss'`.
- `patience` : Cuántas épocas esperar antes de detener el entrenamiento si no hay mejora.
- `restore_best_weights` : Si es `True`, restaura los pesos del modelo con el mejor valor obtenido.

2. `ReduceLROnPlateau`

```
lr_scheduler = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=3,
    min_lr=1e-7
)
```

Reduce **automáticamente la tasa de aprendizaje** cuando una métrica deja de mejorar.

Permite al modelo **seguir mejorando** cuando se estanca, ajustando la tasa de aprendizaje dinámicamente.

Parámetros:

- **monitor** : Métrica a monitorizar.
- **factor** : Por cuánto se multiplica la tasa de aprendizaje actual (ej: **0.5 → reduce a la mitad**).
- **patience** : Número de épocas sin mejora antes de reducir la tasa.
- **min_lr** : Límite inferior para la tasa de aprendizaje.

Otros callbacks útiles que se pueden añadir

3. **ModelCheckpoint** – Guardar el mejor modelo

```
from tensorflow.keras.callbacks import ModelCheckpoint

checkpoint = ModelCheckpoint(
    'mejor_modelo.h5',
    monitor='val_loss',
    save_best_only=True,
    mode='min'
)
```

Guarda automáticamente el modelo durante el entrenamiento, especialmente útil para guardar el **mejor modelo encontrado**.

Se puede reanudar el entrenamiento o usar solo el mejor modelo sin tener que reentrenarlo.

4. **TensorBoard** – Visualización avanzada

```
from tensorflow.keras.callbacks import TensorBoard
import datetime

log_dir = "logs/fit/" + datetime.datetime.now().strftime("%Y%m%d-%H%M%S")
tensorboard = TensorBoard(log_dir=log_dir, histogram_freq=1)
```

Después del entrenamiento, en otra celda, ejecutar TensorBoard

```
%load_ext tensorboard
%tensorboard --logdir logs/fit
```

Una vez ejecutado `%tensorboard --logdir logs/fit`, verás un panel integrado en la celda con pestañas como:

- **SCALARS**: Muestra gráficos de pérdida (`loss`) y métricas (`accuracy` , etc.) por época.
- **GRAPHS**: Muestra la arquitectura del modelo y cómo se conectan las operaciones.
- **HISTOGRAMS**: Muestra cómo cambian las distribuciones de los pesos y sesgos (`weights`).
- **IMAGES** (si usaste): Visualización de imágenes generadas o transformadas.
- **PROJECTOR**: Útil si trabajas con embeddings (vectores de palabras, etc.).

Ideal para **diagnosticar y entender el comportamiento del modelo**.

5. **CSVLogger** – Guardar logs en archivo CSV

Guarda las métricas de entrenamiento en un archivo `.csv`.

Útil para análisis posterior, comparar experimentos o automatizar reportes.

```
from tensorflow.keras.callbacks import CSVLogger

csv_logger = CSVLogger('training.log', append=True)
```

6. **LearningRateScheduler** – Programador personalizado de LR

Permite definir una **función personalizada** para cambiar la tasa de aprendizaje en cada época.

Control total sobre cómo cambia la tasa de aprendizaje (útil en fine-tuning o entrenamientos largos).

```
from tensorflow.keras.callbacks import LearningRateScheduler

def scheduler(epoch, lr):
    if epoch < 10:
        return lr
    else:
        return lr * tf.math.exp(-0.1)

lr_callback = LearningRateScheduler(scheduler)
```

7. **TerminateOnNaN** – Detener si aparece NaN

Detiene el entrenamiento si algún valor **se vuelve NaN** (por ejemplo, por exploding gradients).

Previenes entrenamientos inválidos y errores posteriores difíciles de diagnosticar.

```
from tensorflow.keras.callbacks import TerminateOnNaN
```

```
nan_terminator = TerminateOnNaN()
```

Resumen de callbacks y su uso

Callback	Propósito	Útil para
<code>EarlyStopping</code>	Detener entrenamiento si no mejora	Evitar overfitting
<code>ReduceLROnPlateau</code>	Reducir LR si se estanca	Mejor convergencia
<code>ModelCheckpoint</code>	Guardar modelo automáticamente	Usar el mejor modelo
<code>TensorBoard</code>	Visualización interactiva	Diagnóstico y análisis
<code>CSVLogger</code>	Guardar logs en CSV	Análisis posterior
<code>LearningRateScheduler</code>	Control manual del LR	Estrategias avanzadas
<code>TerminateOnNaN</code>	Detener si hay valores inválidos	Prevenir fallos